REAL-TIME DESIGN SECTION

WebSocket Conceptual Architecture

Interactive Cyber Threat Intelligence Dashboard | Project 8

Meryl & Shambeline | Group N | Landmark Technologies Boot Camp | 2026

1. What Are WebSockets?

WebSockets are a communication protocol that provides a persistent, full-duplex (two-way) connection between a client (web browser) and a server over a single TCP connection. Unlike traditional HTTP, which follows a request-response cycle where the client must always initiate contact, WebSockets allow the server to push data to the client at any time without a new request being made.

The WebSocket protocol was standardized by the IETF as RFC 6455 in 2011 and is natively supported by all modern browsers. A WebSocket connection begins as an HTTP request that is then "upgraded" to a persistent WebSocket connection through a process called the WebSocket handshake.

WebSocket Handshake — Client Request

```
GET /socket HTTP/1.1
Host: cti-dashboard.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlc2Fs...
Sec-WebSocket-Version: 13
```

Server Response — Connection Established

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPL...
```

✓Connection is now persistent and bidirectional

2. How WebSockets Differ from Polling

The current implementation of this CTI Dashboard uses a polling mechanism — JavaScript setInterval() calls the Flask backend every 60 seconds to fetch updated threat data. While this approach is functional and appropriate for a prototype, it has fundamental limitations in a production threat intelligence environment where timing is critical.

Dimension	Polling (Current)	WebSockets (Production)
Connection type	New HTTP request every 60s — connection opens and closes repeatedly	**Single persistent TCP connection — remains open for entire session**
Data direction	Client asks → Server responds (one-way initiation)	**Server pushes → Client receives instantly (true bidirectional)**
Latency	Up to 60 seconds delay between event and display	**Sub-second — data appears on dashboard the moment it is detected**
Server load	High — handles hundreds of repeated HTTP requests per minute	**Low — one connection per client; server pushes once to all**

Network overhead	High — full HTTP headers sent with every request	**Low — minimal framing overhead after initial handshake**
Use case fit	Adequate for prototype and academic demonstration	**Required for production SOC environments where speed is critical**
Implementation	setInterval() + fetch() — already implemented	**socket.io library on Flask backend + client**

3. Production Real-Time CTI Dashboard Using WebSockets

In a production deployment of this Cyber Threat Intelligence Dashboard, WebSockets would replace the polling mechanism entirely. The socket.io library — which wraps the native WebSocket API with fallback support and room-based broadcasting — would be used on both the Flask backend and the browser frontend.

3.1 Backend Implementation (Flask + socket.io)

The production backend would use Flask-SocketIO, a Python library that integrates socket.io into the Flask framework. The server would maintain a persistent connection pool of all connected dashboard clients and broadcast threat updates to all of them simultaneously the moment new data arrives from the AlienVault OTX API.

```
# Flask Backend — WebSocket Implementation (Python)
    from flask import Flask
    from flask_socketio import SocketIO, emit
    from OTXv2 import OTXv2
    import threading, time

    app = Flask(__name__)
    socketio = SocketIO(app, cors_allowed_origins="*")
    otx = OTXv2("YOUR_API_KEY")

    # Background thread — fetches OTX data every 60s
    def threat_broadcast_loop():
        while True:
            pulses = otx.getall(max_items=50)
            threats = [parse_pulse(p) for p in pulses]
            socketio.emit("threat_update", {"data": threats})
            time.sleep(60)

    # Start background thread on server launch
    thread = threading.Thread(target=threat_broadcast_loop)
    thread.daemon = True
    thread.start()
```

3.2 Frontend Implementation (socket.io Client)

The dashboard frontend would connect to the WebSocket server using the socket.io client library. Instead of polling every 60 seconds, the frontend would listen for threat_update events pushed by the server and update all five dashboard panels instantly upon receipt.

```
// Frontend Dashboard — WebSocket Client (JavaScript)
    const socket = io("https://cti-dashboard.com");

    // Listen for server-pushed threat updates
    socket.on("threat_update", function(payload) {
        const threats = payload.data;

        // Update all 5 panels simultaneously
        updateKPICards(threats);
        updateAttackChart(threats);
        updateMalwareChart(threats);
        updateCountriesChart(threats);
        updateTrendChart(threats);
        updateThreatTable(threats);
        updateTimestamp();

        console.log("Live update received:", threats.length, "threats");
    });

    // Handle connection events
    socket.on("connect",    () => console.log("WebSocket connected"));
    socket.on("disconnect", () => console.log("WebSocket reconnecting..."));
```

4. Architecture Diagram — Server Pushing Data to Connected Clients

The following diagram illustrates the complete WebSocket architecture for the production CTI Dashboard, showing how the Flask server acts as the central hub that receives threat data from AlienVault OTX and simultaneously broadcasts updates to all connected analyst clients.

AlienVault OTX API	*Threat feed every 60s → backend ingestion*
Flask Backend + socket.io Server	*Processes + classifies threat data in real time*
WebSocket Channel (ws://)	*Persistent bidirectional connection to all clients*
All Connected CTI Dashboard Clients	*Dashboard panels update instantly — no page reload*

Complete Production Data Flow

1. AlienVault OTX API → Flask backend fetches new threat pulses every 60 seconds
2. Flask backend → Classifies threats into 6 attack types using classify_attack()
3. Flask-SocketIO → Emits threat_update event to ALL connected dashboard clients simultaneously
4. Each client browser → Receives event and updates all 5 panels in under 100ms
5. Connection maintained → Server reconnects automatically if network drops

5. Why WebSockets Matter for Cyber Threat Intelligence

In a Security Operations Centre (SOC) environment, the speed of threat detection and display is a critical operational metric. A delay of 60 seconds between a new threat emerging and it appearing on an analyst's dashboard can mean the difference between early detection and a successful breach. WebSockets eliminate this latency entirely.

Current Prototype (Polling)	Production Target (WebSockets)
✓ Functional for academic demonstration	✓ Sub-second threat display latency
✓ Simple to implement with setInterval()	✓ Single connection per client — low load
✗ Up to 60s delay on threat updates	✓ Server pushes only when new data exists
✗ High server load with many clients	✓ Scales to hundreds of concurrent analysts
✗ Inefficient — fetches even when no new data	✓ Industry standard for SOC dashboards

Design Note

The polling mechanism implemented in this prototype is intentionally designed to be architecturally identical to the WebSocket implementation — the same update functions (updateKPICards, updateAttackChart, etc.) are called in both cases. Migrating from polling to WebSockets in production therefore requires only replacing the setInterval fetch call with a socket.on listener — all panel update logic remains unchanged.

This architectural decision demonstrates forward-thinking software design.